

Maritime CargoService: Sprint 2

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 12, 2026

1. Introduction

Il punto di partenza di questo sprint è il prototipo realizzato nello Sprint 1, documentato al seguente [link](#)^o.

Lo Sprint 1 ha prodotto un prototipo eseguibile del **cargoservice** in cui il ciclo principale di carico viene gestito tramite attori QAK, con componenti simulati per IOPort, sonar, LED e markerdevice, e con il **cargorobot** modellato come wrapper del servizio **robotsmart26**.

L'architettura finale dello Sprint 1, riportata di seguito, costituisce il riferimento di partenza per lo Sprint 2.

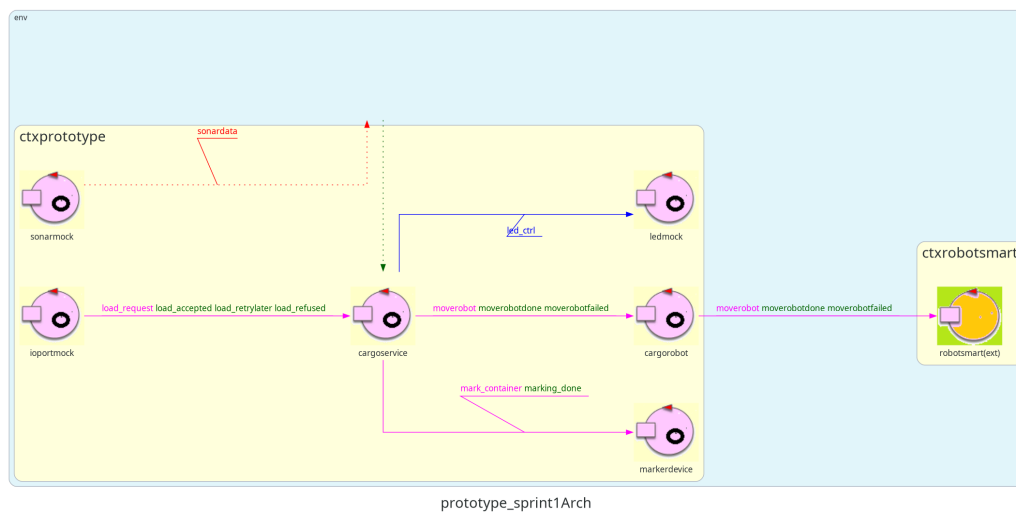


Figure 1: Architettura finale definita nello Sprint 1.

1.1. Goal dello Sprint 2

Il goal dello Sprint 2 è quello definito nella pagina di sintesi dello Sprint 1.

L'obiettivo dello Sprint 2 è evolvere il prototipo sviluppato sostituendo progressivamente i componenti simulati con le rispettive implementazioni reali:

- IOPort come Web GUI;
- sonar e LED collegati al PicoW;
- completare la gestione dello stato **Out of service** integrando il sonar reale, implementando la verifica della misura e l'interruzione/ripresa del movimento del cargorobot.

Si stima una durata di circa **30 ore** complessive di lavoro, corrispondenti a circa 10 ore per ciascun componente del gruppo.

2. Requirements

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o.

Per comodità si riporta anche l'allegato con i requisiti forniti dalla committente.

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the *sonar sensor* measures a distance $D > D_{FREE}$ for at least 3 secs (perhaps a failure of the *sonar*).

Figure 2: Requisiti forniti dalla committente.

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Gli slot1-4 rappresentano le aree della hold riservate per immagazzinare ciascuno un container.
- Lo slot5 rappresenta un'area in cui il cargorobot deve temporaneamente depositare un container, prima di posizionarlo in uno degli slot1-4. Durante la sosta temporanea, un dispositivo 'marker' etichetta il container con un codice a barre identificativo e segnala quando l'attività di marcatura è completata.
- L'IOPort è un dispositivo dotato di un pulsante e di un display. Il pulsante viene premuto dal cliente per inviare una richiesta di carico di un container sulla nave. Il display viene utilizzato per mostrare la risposta alla richiesta e lo stato attuale della hold.
- Il sensore associato all'IOPort è un dispositivo (un sonar) utilizzato per rilevare la presenza di un container, quando misura una distanza D , tale che $D < D_{FREE}/2$, per un tempo ragionevole (ad esempio 3 secondi).
- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.

- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold;
 - il messaggio “**Service working**” quando tutto sta procedendo correttamente;
 - il messaggio “**Out of service**” se il sensore sonar misura la distanza del container dal sonar stesso $D > D_{FREE}$ per almeno 3 secondi.

3. Requirement analysis

L'analisi dei requisiti è stata svolta nello [Sprint 0°](#) e approfondita nello [Sprint 1°](#). Tali risultati vengono assunti come validi anche per lo Sprint 2.

In particolare, per lo Sprint 2 restano centrali le seguenti conclusioni:

- il **cargoservice** è l'orchestratore del ciclo di carico;
- la **hold** rappresenta lo stato interno degli slot e delle posizioni rilevanti;
- il **cargorobot** viene trattato come servizio reattivo, tramite wrapper verso **RobotSmart26**;
- sonar e LED devono essere integrati come dispositivi reali collegati al PicoW;
- l'IOPort deve essere realizzato come Web GUI;
- lo stato **Out of service** deve bloccare l'accettazione di nuove richieste e interrompere il movimento del robot, con ripresa al ritorno dello stato **Service working**.

4. Problem analysis

4.1. Gestione della Hold

Il problema è che molti componentin (da elencare) hanno bisogno di accedere allo stato della hold in tempo reale, ottenendolo in un messaggio “univoco”

Lo stato della hold (attualmente gestito dal tuo POJO Hold) deve quindi qui essere accessibile all'esterno. La scelta migliore è serializzare lo stato in una stringa JSON, poiché è un formato standard, compatibile con qualsiasi protocollo e facilmente manipolabile da interfacce web.

Tuttavia, QAK non offre nativamente la possibilità di attivare un server web integrato. Pertanto, è necessario un componente esterno intermediario.

Si è scelto di condividere la stringa JSON utilizzando il protocollo CoAP (nativamente supportato da QAK). CoAP è ideale perché supporta il meccanismo di observe, permettendo ai client di iscriversi e ricevere notifiche automatiche ai cambiamenti.

4.2. Analisi dell'IOPort come Web GUI

Si sceglie una pagina web nativa html che manda get e p

I browser non supportano direttamente CoAP. Scartiamo il polling (inefficiente) e giustifica l'uso delle WebSocket per inviare aggiornamenti push in tempo reale alla GUI.

Invio della load_request: Invece di usare messaggi QAK diretti o MQTT (che richiederebbero un broker aggiuntivo e complesse configurazioni di sicurezza nel browser), l'utente invierà la richiesta di carico tramite una chiamata HTTP POST dalla GUI verso il server intermediario

4.3. Analisi dell'integrazione sonar e LED su PicoW

Nello Sprint 1 il sonar era un componente simulato la cui unica responsabilità era fungere da “misuratore e trasmettitore” di informazioni, emettendo i dati sotto forma di evento QAK che il cargoservice intercettava. Ora che stai passando al dispositivo fisico, sorge il problema di come far comunicare un componente hardware esterno con l'attore QAK.

Integrazione tra linguaggi e piattaforme diverse: Il cargoservice (scritto in QAK) opera su una Java Virtual Machine, mentre il sonar è gestito da uno script Python su un Raspberry Pi. Come evidenziato nell'analisi dell'altro gruppo, l'adozione di un protocollo standard come MQTT rende il sistema indipendente dal linguaggio, favorendo in modo nativo l'estensione verso componenti scritti in Python, Node.js, ecc.. MQTT agisce quindi come un “ponte” perfetto e universale.

Mappatura naturale con la logica ad Eventi (Publish/Subscribe): Il protocollo MQTT è orientato ai messaggi e si basa su un modello publish/subscribe. Questo paradigma si mappa in modo 1-a-1 con il concetto di Event in QAK. Lo script Python sul Raspberry si limiterà a pubblicare (publish) le misurazioni della distanza su uno specifico topic MQTT. L'infrastruttura di base di QAK, che supporta MQTT nativamente, convertirà automaticamente i messaggi su quel topic in eventi broadcast sonardata, mantenendo la logica del tuo cargoservice completamente inalterata rispetto allo Sprint 1.

Leggerezza e vocazione IoT: MQTT è un protocollo leggero, asincrono e scalabile. È stato progettato specificamente per l'Internet of Things (IoT) e per dispositivi con risorse limitate che devono trasmettere flussi continui di dati (come le misurazioni costanti di un sensore a ultrasuoni).

Vantaggi: Protocollo leggero e asincrono, pienamente supportato dal runtime QAK, perfetta compatibilità con script Python e netta separazione delle responsabilità (il sonar non deve sapere a chi sta inviando i dati, si limita a pubblicarli).

4.4. Gestione dello stato Out of service

Modifiche al tuo cargoservice QAK: Nel tuo codice QAK, dovrai semplicemente aggiungere l'istruzione `updateResource(JSON_STATE)` ogni volta che lo stato della stiva cambia (es. quando prenoti uno slot, quando il container viene fisicamente depositato, o quando il sistema va in Out of service)

4.5. Evoluzione dell'architettura QAK

Nota: Da completare con le modifiche ai messaggi, agli attori e ai contesti rispetto al prototipo dello Sprint 1.

5. Project

5.1. Struttura del progetto

Nota: Da completare con la struttura finale dei moduli e con i riferimenti ai sorgenti implementati nello Sprint 2.

5.2. Implementazione dell'IOPort

Nota: Da completare.

5.3. Implementazione dei dispositivi reali

Nota: Da completare.

5.4. Implementazione della gestione Out of service

Nota: Da completare.

6. Test plans

Nota: Da completare con i test funzionali e di integrazione previsti per lo Sprint 2.

6.1. Test IOPort

Nota: Da completare.

6.2. Test sonar e LED

Nota: Da completare.

6.3. Test Out of service e ripresa del robot

Nota: Da completare.

7. Deployment

Nota: Da completare con le istruzioni di avvio dei moduli, della Web GUI, del PicoW e dei servizi necessari.

8. Maintenance

Nota: Da completare con eventuali note di manutenzione, limiti noti e attività rinviate agli sprint successivi.

9. Pagina di sintesi

9.1. Architettura finale dello Sprint 2

Nota: Da completare con l'architettura finale prodotta nello Sprint 2.

9.2. Goal dello sprint successivo

Nota: Da completare al termine dello Sprint 2.

10. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340